

Assignment: Loops in C Programming

1. Program to Print Numbers from 10 Down to 1 Using a For Loop

```
#include <stdio.h>

int main()
{
    int i;

    for(i = 10; i >= 1; i--)
    {
        printf("%d\n", i);
    }

    return 0;
}
```

Output:

```
10
9
8
7
6
5
4
3
2
1
```

2. Menu-Driven Console App Using While Loop

```
#include <stdio.h>
```

```
#include <string.h>

int main()
{
    char team1[50] = "Mumbai Indians";
    char team2[50] = "Chennai Super Kings";
    char team3[50] = "Royal Challengers Bengaluru";
    char newTeam[50];

    int choice = 0;

    while(choice != 3)
    {
        printf("\n=== IPL Team Menu ===\n");
        printf("1. View Favorite Teams\n");
        printf("2. Add New Team\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        if(choice == 1)
        {
            printf("\nFavorite IPL Teams:\n");
            printf("1. %s\n", team1);
            printf("2. %s\n", team2);
            printf("3. %s\n", team3);
```

```

    }

    else if(choice == 2)

    {

        printf("Enter new team name: ");

        scanf(" %[^\\n]", newTeam);

        printf("New Team Added: %s\\n", newTeam);

    }

    else if(choice == 3)

    {

        printf("Exiting Program...\\n");

    }

    else

    {

        printf("Invalid Choice!\\n");

    }

}

return 0;

}

```

3. "Guess the Song" Game Using Do-While Loop

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
char correctSong[] = "Believer";
char guess[50];

printf("Guess the Song!\n");

do
{
    printf("Enter song name: ");
    scanf(" %[^\\n]", guess);

    if(strcmp(guess, correctSong) != 0)
    {
        printf("Wrong Guess! Try Again.\n");
    }

} while(strcmp(guess, correctSong) != 0);

printf("Correct! The song was %s.\n", correctSong);

return 0;
}
```

Possible Song List:

- Believer
 - Shape of You
 - Blinding Lights
-

4. Difference Between Entry-Controlled and Exit-Controlled Loops

Entry-Controlled Loop (While Loop)

The condition is checked before entering the loop. If the condition is false initially, the loop body will not execute even once.

Example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int x = 5;
```

```
    while(x < 5)
```

```
    {
```

```
        printf("Hello\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Output:

(No Output)

Explanation:

Since $x < 5$ is false at the beginning, the loop never executes.

Exit-Controlled Loop (Do-While Loop)

The condition is checked after executing the loop body. Therefore, the loop executes at least one time even if the condition is false initially.

Example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int x = 5;
```

```
    do
```

```
    {
```

```
        printf("Hello\n");
```

```
    }
```

```
    while(x < 5);
```

```
    return 0;
```

```
}
```

Output:

Hello

Explanation:

The do-while loop executes once before checking the condition, so "Hello" is printed even though $x < 5$ is false.

Conclusion

This assignment demonstrated the use of for loops, while loops, do-while loops, menu-driven programs, simple games, and the difference between entry-controlled and exit-controlled loops in C programming.